

The Backend Engineer Roadmap

From JavaScript Fundamentals to Production-Grade Systems

A step-by-step, fundamentals-first path to becoming a real backend engineer — not just someone who can build a CRUD API — built for self-study through official documentation, books, and YouTube.

Prepared for Shahruk | September 2026

How to Use This Roadmap

Philosophy: Tools change every few years — Express may be replaced by something else, PostgreSQL may get a competitor, and AI will keep automating boilerplate. What doesn't change is your understanding of *why* systems are built the way they are: how a request actually travels from a browser to a server and back, why databases enforce constraints, why we hash passwords a certain way, why systems fail under load and how to design around it. This roadmap is deliberately "fundamentals-heavy" so that in 5 years, when the specific frameworks have changed, you're still a strong engineer.

Structure: The roadmap has 14 phases. Each phase lists topics in the order you should learn them, a short "why this matters" note for the non-obvious ones, and pointers on where to learn it (official docs / self-study resource / YouTube search term). Phases build on each other — don't skip Phase 1–3 even if you're eager to touch Express and Postgres; the fundamentals make everything after them click faster instead of feeling like memorized incantations.

Pace: Don't treat this as a checklist to rush through. For each topic, aim to reach the point where you can explain it out loud to someone else and rebuild a small example from scratch without copying code. Build something — even something tiny — with every phase. Reading/watching without building is the single biggest reason self-taught roadmaps fail to produce real skill.

Resource legend used throughout:

- **[DOCS]** — official documentation (always your primary, most trustworthy source)
- **[BOOK/ARTICLE]** — a well-known book or long-form written resource
- **[YT SEARCH]** — a search phrase to use on YouTube (channels change; searching by topic keeps you future-proof)
- **[PRACTICE]** — something to build or a site to practice on

Phase	Focus	Rough Duration*
0	JavaScript refreshed for backend use	1 week
1	Computer science & programming fundamentals	2–3 weeks
2	Operating systems, Linux, networking & the internet	2 weeks
3	Node.js runtime deep dive	2 weeks
4	HTTP & API design fundamentals	1–2 weeks
5	Express.js & server architecture	2 weeks
6	Relational databases, SQL & PostgreSQL	3–4 weeks
7	Caching & NoSQL (Redis, MongoDB)	1–2 weeks
8	Authentication, authorization & security	2–3 weeks
9	Testing & code quality	1–2 weeks
10	Containers, CI/CD & deployment	2–3 weeks
11	System design & scalability	3–4 weeks
12	Observability & production operations	1–2 weeks
13	Advanced backend: queues, microservices, GraphQL/gRPC	2–3 weeks
14	Backend engineering in the AI era	2 weeks + ongoing

*Durations assume steady part-time study; treat them as a compass, not a deadline. Total $\approx$ 7–9 months to a strong junior/mid-level backend engineer level, with lifelong depth after that.

Phase 0 — JavaScript, Refreshed for Backend Use

You already know JS basics. This phase fills the specific gaps that matter for server-side code.

0.1 Asynchronous JavaScript, properly understood

- The event loop, call stack, microtask queue vs macrotask queue — not just "async/await is nice syntax," but what actually happens under the hood
- Callbacks → Promises → async/await, and how to convert between them
- Promise combinators: `Promise.all`, `allSettled`, `race`, `any`
- Error handling in async code: `try/catch` with `await`, unhandled promise rejections

Why this matters: Every backend request in Node.js is asynchronous under the hood (DB calls, file reads, network calls). If you don't deeply understand the event loop, you will eventually write code that silently drops errors, blocks the server, or executes in an order you didn't expect — and you won't know why.

0.2 Modern JS constructs used constantly in backend code

- Destructuring, spread/rest, optional chaining, nullish coalescing
- Modules: CommonJS (`require`) vs ES Modules (`import`), and why Node supports both
- Closures and higher-order functions (used everywhere in middleware)
- `this` binding, arrow functions vs regular functions
- Classes, prototypes, and prototypal inheritance (Node's internals and many libraries use this)

0.3 TypeScript fundamentals

- Why static typing matters for backend systems that live for years and are touched by many people
- Basic types, interfaces vs types, generics, union/intersection types
- `tsconfig.json` essentials, compiling TS for Node

[DOCS] developer.mozilla.org (MDN) for JS fundamentals | typescriptlang.org/docs for TypeScript

[YT SEARCH] "JavaScript event loop explained," "TypeScript for beginners full course"

Milestone: Write a small Node script that reads a file, calls a public API, and processes both results together using `async/await` and `Promise.all` — without any `callback-hell`.

Phase 1 — Computer Science & Programming Fundamentals

The foundation that makes you an engineer instead of a framework-user.

1.1 Data structures

- Arrays, linked lists, stacks, queues
- Hash tables/maps (understand how they achieve $O(1)$ lookup — this explains a huge amount about databases and caching later)
- Trees (binary trees, BSTs), graphs (basic traversal: BFS/DFS)

1.2 Algorithms & complexity

- Big-O notation — time and space complexity
- Sorting and searching algorithms (at least conceptually: merge sort, quicksort, binary search)
- Recursion

Why this matters: You don't need to be a competitive programmer to be a good backend engineer, but you do need to reason about why one query plan or one algorithm is faster than another, why an $O(n^2)$ loop over a large dataset will melt your server, and how indexes turn $O(n)$ lookups into $O(\log n)$.

1.3 Core software engineering principles

- DRY, KISS, YAGNI, separation of concerns
- SOLID principles (especially Single Responsibility and Dependency Inversion — these show up constantly in backend architecture)
- Basic design patterns: Singleton, Factory, Repository, Strategy, Observer
- Clean code practices: naming, function size, avoiding deep nesting

1.4 Version control (deepen beyond basics)

- Branching strategies (feature branches, Git Flow basics)
- Rebasing vs merging, resolving conflicts
- Writing good commit messages, using pull requests effectively

[BOOK] "Grokking Algorithms" by Aditya Bhargava (very approachable) | "Clean Code" by Robert C. Martin

[PRACTICE] NeetCode.io or LeetCode (Easy tier) for DS&A — you're already doing this daily, keep it going alongside this roadmap

[YT SEARCH] "SOLID principles explained," "Big O notation explained simply"

Milestone: Implement a hash map and a singly linked list from scratch in TypeScript, without looking up the implementation — just the concept.

Phase 2 — Operating Systems, Linux & Networking

Backend code doesn't run in a vacuum — it runs on a machine, talks over a network, and lives on a server you'll manage.

2.1 Operating system fundamentals

- Processes vs threads, what "concurrency" and "parallelism" actually mean
- Memory: stack vs heap, garbage collection basics
- File systems and file descriptors (Node treats almost everything — files, sockets — as a stream/descriptor)
- Environment variables and how processes read configuration

2.2 Linux & the command line

- Navigating the filesystem, permissions (chmod/chown), processes (ps, top, kill)
- Package managers (apt), SSH into a remote machine
- Shell scripting basics (bash) — enough to automate small tasks
- Environment/config files: .env, .bashrc, PATH

Why this matters: Almost every real backend runs on Linux. If a production server is having issues, you need to be comfortable SSHing in, reading logs, and checking process/resource usage without panicking.

2.3 Networking fundamentals

- The OSI model (conceptually) and where HTTP, TCP, IP sit in it
- TCP vs UDP, the three-way handshake, what a "socket" is
- DNS: how a domain name resolves to an IP address
- IP addresses, ports, what "localhost:3000" actually means
- TLS/SSL basics — how HTTPS encryption works at a high level

[DOCS] "Linux Journey" (linuxjourney.com) — free, structured, beginner-friendly

[YT SEARCH] "networking fundamentals for developers," "how DNS works," "TCP vs UDP explained," "how HTTPS works"

Milestone: Rent or spin up a free-tier Linux VM (e.g., Oracle Cloud/AWS free tier), SSH into it, and run a simple Node "Hello World" script on it manually.

Phase 3 — Node.js Runtime, Deeply

Not "how to use Express" yet — how Node itself actually works.

3.1 Node's architecture

- The V8 engine, libuv, and the event loop phases (timers, I/O callbacks, poll, check, close)
- Single-threaded JS execution + the thread pool for I/O-heavy operations
- Blocking vs non-blocking code — why `fs.readFileSync` is dangerous in a server

3.2 Core Node modules (no frameworks yet)

- Build a raw HTTP server using only the built-in `http` module — parse the request, write a response manually
- `fs` (file system), `path`, `events` (`EventEmitter`), `stream`
- Streams and buffers — how Node handles large data (file uploads, video) without loading everything into memory

Why this matters: Express is a thin layer over Node's `http` module. If you build a raw server first, Express stops feeling like magic — you'll immediately understand what a "middleware" is, because you'll have manually written the equivalent of one.

3.3 Package management & project structure

- `npm/pnpm`: `package.json`, semantic versioning, lockfiles, dependencies vs `devDependencies`
- Environment configuration with `dotenv`
- Structuring a Node project (`config`, `routes`, `controllers`, `services`, `models` — the reasoning behind layering, covered more in Phase 5)

[DOCS] nodejs.org/docs (official Node.js docs — read the Event Loop guide specifically)

[YT SEARCH] "Node.js event loop deep dive," "build an HTTP server without Express"

Milestone: Build a working HTTP server with zero dependencies that can serve a JSON API for a simple "notes" resource (GET/POST/DELETE), using only Node's built-in `http` module.

Phase 4 — HTTP & API Design Fundamentals

Understand the protocol and the design philosophy before touching a framework that hides it.

4.1 HTTP protocol, in depth

- Request/response structure: headers, status lines, body
- HTTP methods and their real semantics (idempotency of GET/PUT/DELETE vs POST)
- Status codes — not just memorizing 200/404/500, but understanding the categories (1xx-5xx) and picking the correct one
- Headers that matter: Content-Type, Authorization, Cache-Control, CORS-related headers
- Cookies vs headers vs query params vs body — when to use each

4.2 API design philosophy

- REST principles (statelessness, resource-based URLs, uniform interface) — and where real-world APIs bend the "pure" rules
- Designing clean resource URLs and versioning strategies (/api/v1/ . . .)
- Pagination, filtering, sorting conventions
- Idempotency keys for safe retries
- Request validation and consistent error response shapes

Why this matters: A framework will happily let you build an inconsistent, hard-to-use API. Understanding REST as a design philosophy (not just a syntax) is what separates an API that's a pleasure to consume from one that generates support tickets.

4.3 API documentation & contracts

- OpenAPI/Swagger specification basics
- Why a documented contract matters for frontend/backend collaboration

[DOCS] developer.mozilla.org/en-US/docs/Web/HTTP (MDN's HTTP docs are excellent and official-adjacent)

[PRACTICE] Postman or Insomnia to manually craft and inspect HTTP requests

[YT SEARCH] "REST API design best practices," "HTTP status codes explained properly"

Milestone: Design (on paper first) a REST API spec for a "blog" resource — routes, methods, status codes, error format — before writing a single line of code.

Phase 5 — Express.js & Server Architecture

Now the framework makes sense, because you already understand what it's automating.

5.1 Express fundamentals

- Routing, route parameters, query strings
- Middleware — what it is, the request/response/next cycle, writing your own
- Error-handling middleware and centralizing error responses
- Serving static files, body parsing, CORS configuration

5.2 Application architecture

- Layered architecture: routes → controllers → services → repositories/models
- Why separating "business logic" from "HTTP handling" matters (testability, reusability)
- Dependency injection basics (even simple manual DI, not necessarily a framework)
- Configuration management for different environments (dev/staging/prod)

Why this matters: Most beginner Express apps cram database queries directly into route handlers. It works for a toy project and becomes unmaintainable fast. Learning proper layering now is the difference between code you can extend for years and code you rewrite every 3 months.

5.3 Input validation & error handling

- Schema validation libraries (e.g., Zod or Joi) and why you never trust client input
- Centralized error handling and custom error classes
- Logging basics (structured logs, log levels)

[DOCS] expressjs.com (official guide + API reference)

[YT SEARCH] "Express.js middleware explained," "Node.js project structure best practices," "MVC pattern Node.js"

Milestone: Rebuild the Phase 3 "notes" API in Express with a proper layered structure, input validation, and centralized error handling.

Phase 6 — Relational Databases, SQL & PostgreSQL

The heaviest phase — databases are where most real backend engineering skill lives.

6.1 Relational database theory

- What a relational model actually is: tables, rows, columns, keys
- Primary keys, foreign keys, and referential integrity
- Normalization (1NF, 2NF, 3NF) — and when to deliberately denormalize
- ACID properties (Atomicity, Consistency, Isolation, Durability) — explained conceptually with real examples

6.2 SQL, from fundamentals to advanced

- SELECT, WHERE, ORDER BY, GROUP BY, HAVING
- JOINS — inner, left, right, full — and why each exists
- Subqueries and Common Table Expressions (CTEs)
- Aggregate functions, window functions
- INSERT/UPDATE/DELETE and transactional safety

6.3 PostgreSQL specifically

- Data types (including JSONB — Postgres's hybrid relational/document capability)
- Constraints (UNIQUE, CHECK, NOT NULL, FOREIGN KEY with ON DELETE behavior)
- Indexes: B-tree indexes and when a query actually benefits from one
- EXPLAIN ANALYZE — reading a query plan to understand why a query is slow
- Transactions: BEGIN/COMMIT/ROLLBACK, isolation levels, understanding race conditions

Why this matters: An engineer who can write a SELECT statement is common. An engineer who can look at a slow query, read its execution plan, and know exactly which index will fix it — that's the skill that gets you hired and trusted with production systems.

6.4 Talking to Postgres from Node

- Raw SQL via the pg driver first (don't skip this — understand the raw layer)
- Connection pooling and why it matters under load
- Then an ORM/query builder (Prisma or Drizzle) — and understanding what it's generating for you under the hood
- Migrations: why schema changes need to be version-controlled and repeatable

[DOCS] [postgresql.org/docs](https://www.postgresql.org/docs/) (the official docs are genuinely excellent and worth reading directly)

[PRACTICE] pgexercises.com (free, hands-on SQL exercises against a real dataset)

[YT SEARCH] "SQL joins explained," "database normalization explained," "PostgreSQL indexing explained," "Prisma ORM crash course"

Milestone: Design a normalized schema (3+ related tables with proper foreign keys) for a small app (e.g., a blog with users, posts, comments), write it in raw SQL, then re-implement the data layer with an ORM and compare the generated queries.

Phase 7 — Caching & NoSQL

Knowing when NOT to use a relational database is as important as knowing SQL.

7.1 Caching fundamentals

- Why caching exists — trading staleness for speed
- Cache invalidation strategies (this is famously one of the hardest problems in CS — understand why)
- Redis: key-value basics, TTLs (expiry), common data structures (strings, hashes, lists, sets)
- Where caching fits in an architecture (application-level cache vs CDN vs DB query cache)

7.2 NoSQL, conceptually and practically

- Document databases (MongoDB) — when a flexible schema genuinely helps vs when it's a shortcut that causes pain later
- CAP theorem (Consistency, Availability, Partition tolerance) at a conceptual level
- Basic MongoDB CRUD and when to reach for it over Postgres

Why this matters: A common junior mistake is picking a database because it's trendy, not because it fits the data's shape and access patterns. Understanding the trade-offs lets you make that call deliberately.

[DOCS] redis.io/docs, mongodb.com/docs

[YT SEARCH] "Redis crash course," "SQL vs NoSQL explained," "cache invalidation strategies"

Milestone: Add a Redis cache layer in front of one expensive query in your Phase 6 project, and measure the response time difference.

Phase 8 — Authentication, Authorization & Security

The phase most tutorials rush — don't rush it.

8.1 Authentication

- Password hashing — why plaintext/reversible encryption is unacceptable, how bcrypt/argon2 work conceptually (salting, work factor)
- Session-based auth (cookies + server-side sessions) vs token-based auth (JWT)
- JWT structure (header/payload/signature), signing, expiry, refresh tokens
- OAuth2 / "Sign in with Google" flow, at a conceptual level

8.2 Authorization

- Role-based access control (RBAC) vs permission-based access control
- Middleware patterns for protecting routes

8.3 Web security fundamentals

- OWASP Top 10 — at least understand each one conceptually: SQL injection, XSS, CSRF, broken auth, etc.
- Why parameterized queries prevent SQL injection (tie this back to Phase 6)
- Input sanitization, rate limiting, and brute-force protection
- HTTPS/TLS in practice, secure cookie flags (HttpOnly, Secure, SameSite)
- Secrets management — never hardcoding API keys/credentials

Why this matters: Security isn't a "nice to have" feature you bolt on later — a single SQL injection or a badly implemented auth system can end a company or leak users' data. This is one of the areas where "I'll learn it if I need it" fails badly, because by the time you need it, the damage is done.

[DOCS] owasp.org/www-project-top-ten

[YT SEARCH] "JWT explained," "how bcrypt works," "OWASP top 10 explained," "OAuth2 explained simply"

Milestone: Add full auth (signup/login with hashed passwords + JWT) and role-based route protection to your project; then deliberately try to break your own login form (SQL injection attempt, wrong-token attempt) to confirm your defenses work.

Phase 9 — Testing & Code Quality

The habit that separates hobby code from professional code.

9.1 Testing fundamentals

- Why we test: catching regressions, enabling confident refactors
- The testing pyramid: unit tests, integration tests, end-to-end tests — and the right ratio between them
- Unit testing with Jest or Vitest — assertions, mocking, test doubles
- Integration testing an Express API (spinning up a test DB, using supertest)

9.2 Code quality tooling

- ESLint and Prettier — automated style/consistency enforcement
- Git hooks (Husky) to enforce checks before commits
- Code review practices — what a good PR review actually looks for

Why this matters: In a team, "it works on my machine" isn't good enough. Tests are how you (and future teammates, and AI coding assistants) can change code without breaking things silently.

[DOCS] jestjs.io/docs

[YT SEARCH] "Jest testing crash course," "testing Node.js APIs with supertest," "testing pyramid explained"

Milestone: Write unit tests for your service layer and integration tests for your API routes, aiming for meaningful coverage of core logic (not just chasing a % number).

Phase 10 — Containers, CI/CD & Deployment

Getting your code from your laptop to the real world, reliably and repeatably.

10.1 Docker

- Why containers exist — "works on my machine" solved via reproducible environments
- Images vs containers, writing a Dockerfile, layers and caching
- Docker Compose for multi-service local dev (app + Postgres + Redis together)

10.2 CI/CD

- What continuous integration and continuous deployment actually mean and solve
- GitHub Actions basics — running tests automatically on every push
- Automated deployment pipelines (build → test → deploy)

10.3 Deployment & cloud basics

- Difference between PaaS (Render, Railway), IaaS (raw AWS EC2/DigitalOcean droplets), and serverless
- Reverse proxies (Nginx) and why they sit in front of Node apps in production
- Environment variables and secrets in production
- Zero-downtime deploys, basic load balancing concepts

[DOCS] docs.docker.com, docs.github.com/actions

[YT SEARCH] "Docker for beginners full course," "GitHub Actions CI/CD tutorial," "Nginx reverse proxy explained"

Milestone: Containerize your full project (app + Postgres + Redis) with Docker Compose, set up a GitHub Actions pipeline that runs your tests on every push, and deploy it to a real public URL.

Phase 11 — System Design & Scalability

Thinking about systems, not just endpoints.

11.1 Scaling fundamentals

- Vertical vs horizontal scaling
- Stateless vs stateful services — why statelessness enables horizontal scaling
- Load balancing strategies (round robin, least connections)
- Database scaling: read replicas, sharding (conceptually)

11.2 Distributed systems concepts

- Consistency vs availability trade-offs (revisit CAP theorem, deeper this time)
- Eventual consistency, idempotency in distributed retries
- Message queues (RabbitMQ or a cloud equivalent) and why async processing decouples systems
- Rate limiting strategies (token bucket, sliding window)

11.3 System design practice

- How to approach a system design interview/exercise: clarify requirements → estimate scale → high-level design → deep dive → trade-offs
- Classic exercises to practice: design a URL shortener, design a rate limiter, design a notification system, design a simplified Twitter feed

Why this matters: This is the phase that turns "I can build features" into "I can be trusted to design a system that will still work at 100x the current traffic." It's also the single most-tested skill in mid/senior backend interviews.

[BOOK] "Designing Data-Intensive Applications" by Martin Kleppmann (the canonical, must-read book once you're through Phases 1-10)

[YT SEARCH] "system design interview basics," "designing a URL shortener," "message queues explained"

[PRACTICE] [systemdesign.one](#) or "System Design Primer" GitHub repo

Milestone: Write a one-page system design document (requirements, high-level diagram, trade-offs) for scaling your own project to handle 1 million users, even if you never build it.

Phase 12 — Observability & Production Operations

How you know a production system is actually healthy — and how you find out fast when it isn't.

12.1 Logging & monitoring

- Structured logging (JSON logs) vs plain text, and log levels (debug/info/warn/error)
- Centralized log aggregation (conceptually — e.g., how tools like Grafana Loki or the ELK stack fit in)
- Metrics: what to measure (latency, error rate, throughput — the "RED" method: Rate, Errors, Duration)

12.2 Alerting & incident response

- Health checks and uptime monitoring
- Basic alerting thresholds and why alert fatigue is a real problem to design against
- What a post-mortem/incident report looks like and why blameless post-mortems matter

[YT SEARCH] "observability vs monitoring explained," "structured logging Node.js," "SRE golden signals"

Milestone: Add structured logging and a /health endpoint to your project, and set up a free uptime monitor (e.g., UptimeRobot) pointing at your deployed app.

Phase 13 — Advanced Backend Patterns

Once the fundamentals are solid, broaden into patterns used at larger scale.

13.1 Beyond REST

- GraphQL — what problem it solves relative to REST (over/under-fetching), basic schema and resolver design
- gRPC and Protocol Buffers — when strongly-typed, high-performance service-to-service communication matters
- WebSockets — real-time, bidirectional communication (chat apps, live updates)

13.2 Microservices vs monoliths

- Honest trade-offs — most companies over-adopt microservices too early; understand why a well-structured monolith is often the right call
- Service-to-service communication patterns, API gateways
- Distributed transactions and the Saga pattern (conceptually)

[YT SEARCH] "GraphQL vs REST," "gRPC explained," "monolith vs microservices trade-offs," "WebSockets crash course"

Milestone: Add one real-time feature (e.g., live notifications via WebSockets) to your project.

Phase 14 — Backend Engineering in the AI Era

AI won't replace an engineer who understands systems — it multiplies one. This phase is what makes you future-proof.

14.1 Using AI as a force multiplier, correctly

- Using AI coding assistants (Claude Code, Copilot, etc.) to accelerate boilerplate — while still reviewing and understanding every line before it ships
- Why the fundamentals in Phases 0–13 matter more, not less, in the AI era: AI can generate a working-looking Express route or SQL query instantly, but only a grounded engineer can tell whether it's secure, correctly indexed, or will fail under load
- Effective prompting for technical tasks: giving AI the right context (schema, constraints, existing code style) to get useful output

14.2 Integrating AI capabilities into backend systems

- Calling LLM APIs from a backend service — structuring prompts, handling streaming responses, managing costs/rate limits
- Vector databases and embeddings (conceptually) — how semantic search and RAG (retrieval-augmented generation) actually work
- Designing APIs for AI-powered features: async job patterns for long-running AI calls, idempotency for retries
- Basic agent/tool-use concepts — how an LLM can be given "tools" (functions) it can call, and how you'd design a backend to support that safely

Why this matters: Many products in 2026 and beyond will have an AI feature bolted onto a very ordinary backend (auth, database, APIs — everything from Phases 0–13). The engineers who stand out are the ones who can build the reliable, secure, well-tested "boring" backend AND wire AI capabilities into it responsibly.

14.3 Staying current without chasing every trend

- Follow official release notes/changelogs for Node, PostgreSQL, and your core stack rather than chasing every new framework
- Build a habit of reading source code and RFCs/specs occasionally — it's the fastest way to deeply understand a tool

[DOCS] platform's official API docs for whichever LLM provider you use (read the docs directly, not just tutorials)

[YT SEARCH] "retrieval augmented generation explained," "vector databases explained," "building AI agents backend architecture"

Milestone: Add one AI-powered feature to your capstone project (e.g., AI-generated task summaries, or a semantic search over notes using embeddings) with proper error handling, rate limiting, and cost awareness.

Closing Notes: Portfolio & Job Readiness

Turn what you built into proof

- By the end of this roadmap you should have one substantial project (ideally your TaskForge AI project) that demonstrates: proper architecture, a real relational schema, authentication/security, tests, containerization, CI/CD, and at least one AI-powered feature
- Write a clear README: the "why," the architecture diagram, trade-offs you made and why
- Be ready to explain any line of your project out loud — in interviews, "I understand why I built it this way" matters far more than the tech-stack list

Keep two tracks running in parallel

- This roadmap (breadth + depth of backend engineering)
- Daily DSA/problem-solving practice (which you're already doing) — the two reinforce each other

Final thought: Nobody finishes "learning backend engineering" — even senior engineers are still learning. The goal of this roadmap isn't to reach some finish line; it's to build the habit of asking "why does this work this way?" at every layer, so that whatever comes next in this field, you can learn it quickly and confidently.